

W4

BLOCK 1 · CLASS 4 · PRACTICAL

Rules, judge, loop.

Last week you watched a feature get built. Today you build one alone — with the three standards every team that ships with agents uses.

2026-09-19

Rules, judge, loop

Timing: check-in 5 · what an agent is + tool calls + ReAct/harness + chronology 10 · rulebook + live AGENTS.md 10 · judge 5 · loop 5 · feature 2 kick-off live 13 · your turn 32 · retro 7 · wrap 3 = 90. TA-run. Presenting laptop: Splitlt repo with bugs 000/001 fixed and feature 1 merged, on main, clean; Codex open on it; GitHub Desktop signed in. Reset the data before class. Nothing new to install today — that is the point of the week.

W4

BLOCK 1 · CLASS 4 · PRACTICAL

Rules, judge, loop.

Last week you watched a feature get built. Today you build one alone — with the three standards every team that ships with agents uses.

What we'll do.

- 1 **Check-in** — your GitHub page: feature 1 merged.
- 2 **What an agent is** — a chat model vs an agent, tool calls, the ReAct loop, the harness, and how we got here.
- 3 **Three standards** — the rulebook, the judge, the loop. Then you use them.
- 4 **Feature 2 kick-off, live** — branch → red → the loop prompt → watch it loop → read the diff.
- 5 **Your turn** — one bug, then feature 2. Thirty-two minutes.
- 6 **Retro** — the rule you add to `AGENTS.md` from what went wrong.

Same groups, but every laptop builds in its own repo today.

2026-09-19

Rules, judge, loop

└─What we'll do.

CLASS 4 · TODAY

What we'll do.

- 1 **Check-in** — your GitHub page: feature 1 merged.
- 2 **What an agent is** — a chat model vs an agent, tool calls, the ReAct loop, the harness, and how we got here.
- 3 **Three standards** — the rulebook, the judge, the loop. Then you use them.
- 4 **Feature 2 kick-off, live** — branch → red → the loop prompt → watch it loop → read the diff.
- 5 **Your turn** — one bug, then feature 2. Thirty-two minutes.
- 6 **Retro** — the rule you add to `AGENTS.md` from what went wrong.

Same groups, but every laptop builds in its own repo today.

Show your GitHub page.

- Repository → branches: `feature-1-balances` or `feature-1-overdue` exists.
- A merged pull request, or the feature commit on `main`.
- Not there yet? Finish it during your turn, before feature 2. The TA notes who.

While the TA walks: pull `main` in GitHub Desktop, run
`uv run pytest tests/test_feature_1.py`. Green? You're ready.

2026-09-19

Rules, judge, loop

└─ Show your GitHub page.

Walk the room looking at browsers, not code. Two minutes per row. Anyone without feature 1 merged does it first in the 32 minutes; they still get bug + feature 2 as homework. Don't debug git at the front — send them to the "when git is confusing" slide from last week.

CLASS 4 · CHECK-IN · 5 MIN

Show your GitHub page.

- Repository → branches: `feature-1-balances` or `feature-1-overdue` exists.
- A merged pull request, or the feature commit on `main`.
- Not there yet? Finish it during your turn, before feature 2. The TA notes who.

While the TA walks: pull `main` in GitHub Desktop, run
`uv run pytest tests/test_feature_1.py`. Green? You're ready.

A chat model writes. An agent acts.

A PLAIN LLM — CHATGPT IN A BROWSER

Text in, text out.

It predicts the next words of an answer. It has never seen your folder — it cannot open a file, run a test, or check anything. *You copy code in, you paste it out, you find out whether it works. "The tests pass" is a sentence that sounds right.*

AN AGENT — CODEX ON YOUR REPO

The same model, plus tools, in a loop.

It can **read** your files, **run** commands, **edit** code — and it sees the result of each action before choosing the next. It keeps going until the job is done or it needs you. That is why it asks permission first.

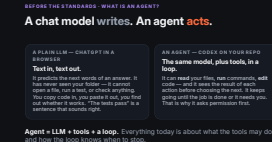
Agent = LLM + tools + a loop. Everything today is about what the tools may do, and how the loop knows when to stop.

2026-09-19

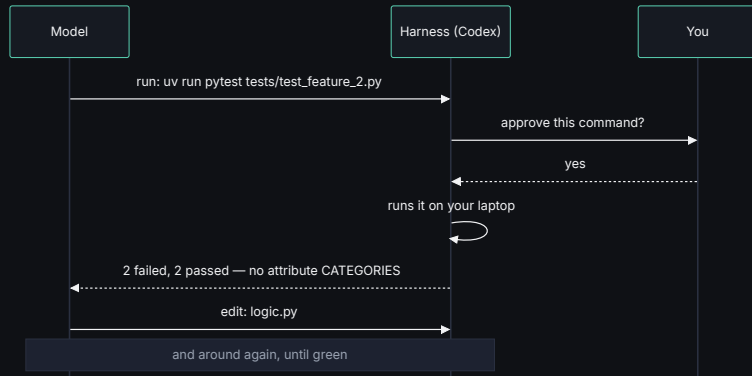
Rules, judge, loop

└ A chat model **writes**. An agent **acts**.

One minute on the contrast, in their own experience: in Class 2 they asked Codex "show me the last line of data/expenses.csv" and it showed the real line — ChatGPT in a browser could not have. Same model family, different powers. The word "agentic" just means "can act, not only answer".



How an agent acts: it asks for a tool.

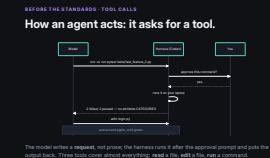


The model writes a **request**, not prose; the harness runs it after the approval prompt and puts the output back. Three tools cover almost everything: **read** a file, **edit** a file, **run** a command.

2026-09-19

Rules, judge, loop

└─ How an agent acts: it asks for a tool.



Tie it to what they will see in ten minutes: on stage, narrate the kick-off as tool calls ("that's a run — that's a read — that's an edit"). The approval prompt is where the human sits inside the loop. And the sting: an agent can run the tests, but it can still **say** they passed without showing you — which is why standard 2 asks for the output, not the sentence. If asked "is this the same model as ChatGPT?" — same family; the difference is tools + loop + your repo, not intelligence.

ReAct: **reason**, **act**, observe. Repeat.



THE HARNESS

The model can only *write* the tool call. Something runs it, shows you the approval prompt, puts the output back, keeps the history, reads `AGENTS.md`, and stops the loop. That software is the **harness**: Codex, Claude Code, Cursor, Copilot's agent.

WHY IT MATTERS

Every agentic coding tool runs this same loop. They differ in the tools they offer and what they ask permission for — not in the loop. Learn the loop once; the tools are interchangeable.

Yao, Zhao, Yu, Du, Shafran, Narasimhan & Cao (2022). *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR 2023 · arxiv.org/abs/2210.03629

2026-09-19

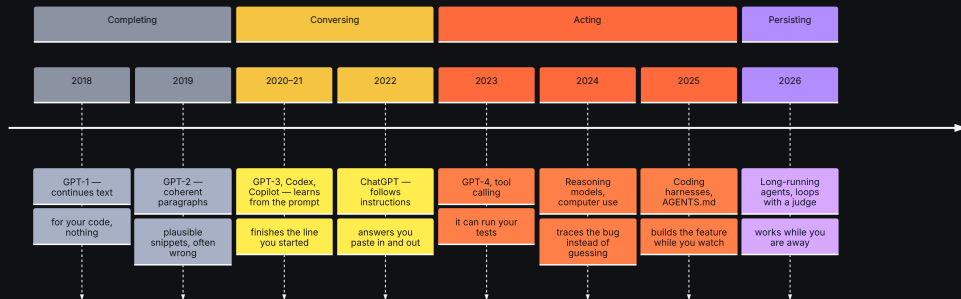
Rules, judge, loop

└ ReAct: **reason**, **act**, observe. Repeat.



ReAct = "Reasoning and Acting", Yao et al. 2022 (ICLR 2023, arXiv:2210.03629): a model does far better when it alternates a thought with an action and reads the observation, instead of answering in one go. Every harness since is that loop with better plumbing. The word to land: harness — model + tools + loop + rules + permissions. When students say "Codex did X", the harness ran the loop; the model chose the actions. Standards 1-3 are you configuring the harness: the rulebook it loads, the judge it runs, the stop condition of its loop.

Eight years, four kinds of capability.



The point. The model got better every year. The job changed twice: when it learned to take **instructions**, and when it got **tools and a loop**. Everything after is plumbing and stamina — and a model that can act for hours needs rules and a judge.

2026-09-19

Rules, judge, loop

└ Eight years, four kinds of capability.



Two minutes, one sweep left to right. Say the four band names as you go — completing, conversing, acting, persisting — that is the shape to remember, not the model names. Reasoning row example to say aloud: a 2023-style model asked why SplitIt's balances sum to +10 pattern-matches a plausible edit; a reasoning model writes out "payer credited 30, two others debited 15 each, payer never debited, so the divisor is 2 and should be 3" and lands on line 95 — what a good student does on paper. Sources if asked: GPT-1 Radford et al. 2018; GPT-2 2019; GPT-3 Brown et al. 2020; Codex Chen et al. 2021; InstructGPT Ouyang et al. 2022 then ChatGPT Nov 2022; ReAct Yao et al. 2022; GPT-4 Mar 2023, function calling Jun 2023; o1 Sep 2024, Claude computer use Oct 2024, DeepSeek-R1 Jan 2025, Operator Jan 2025, Claude 3.7 extended thinking Feb 2025; Claude Code Feb 2025, Codex CLI Apr 2025, agents.md Aug 2025; Codex app Feb 2026. Dates are release months, approximate on purpose.

AGENTS.md — the file it reads first.

WHAT IT IS

A README for agents

Codex reads it at the start of every session, before anything else. So do twenty other tools. Your repo has had one since Class 2.

WHAT GOES IN

Only what it can't guess

Commands (`uv run ...`). Boundaries (*never edit tests/*). Etiquette (*branch named after the spec*). How to report (*paste the test output*).

WHAT STAYS OUT

Anything it can read

Explanations of the code. Tutorials. "Write clean code." Every extra line hides the important ones.

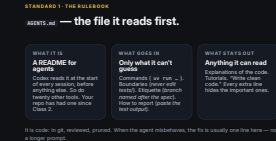
It is code: in git, reviewed, pruned. When the agent misbehaves, the fix is usually one line here — not a longer prompt.

2026-09-19

Rules, judge, loop

AGENTS.md — the file it reads first.

Ask Codex on stage: "What does AGENTS.md tell you to do?" — it recites the file. That is the proof it reads it. The test for every line: would removing it cause a mistake? If not, cut it. Mention the 32 KB cap only if asked; the real cap is attention.



Three lines. One commit.

Open `AGENTS.md` in your repo. Under **How to answer**, add:

- Build each spec on a branch named after it, e.g. feature-2-categories. Never commit to main directly.
- After every change, run the tests for the task and paste the final output. The output is the proof, not a sentence.
- Never edit, delete or skip anything in tests/. If a test looks wrong, say so and stop.

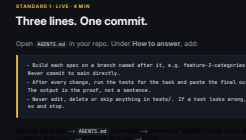
GitHub Desktop → `AGENTS.md` changed → summary *"AGENTS.md: rules for building features"* → Commit to main → Push.

2026-09-19

Rules, judge, loop

└─ Three lines. One commit.

Do it on stage first: ask Codex "Add these three lines under How to answer in AGENTS.md", read the diff, accept. Then commit in GitHub Desktop. Point out the last one already existed in spirit ("never edit tests") — now it is explicit and it is a rule the student enforces. Codex picks the new rules up on the next thread, not mid-thread: say "new thread" now.



Define **done** before you build.

SOMETHING THAT CAN'T LIE

The agent stops when the work *looks* done. Give it a check that says pass or fail: the spec's **Done when** and `tests/test_feature_2.py`. Four red tests are your to-do list.

NEVER MOVE THE GOALPOSTS

An agent that edits a test to make it pass has cheated, not finished. The rule is in `AGENTS.md`. You enforce it.

EVIDENCE, NOT CLAIMS

"All tests pass" is a sentence. The test output is evidence. Ask for it every time.

GREEN IS NOT THE END

The judge checks what it was told to check. After green: read the diff, click the app. App wrong, tests green? Write down what the judge missed — that's a test that should exist.

2026-09-19

Rules, judge, loop

└ Define **done** before you build.

Run the red tests on stage: `uv run pytest tests/test_feature_2.py` gives 4 failed, "logic has no attribute CATEGORIES". Open the test file and match each test to a checklist line in the spec, like last week. The "green is not the end" card matters today: feature 2's selectbox in the form is not tested at all — only clicking the app catches a missing one.

STANDARD 2 · THE JUDGE

Define **done** before you build.

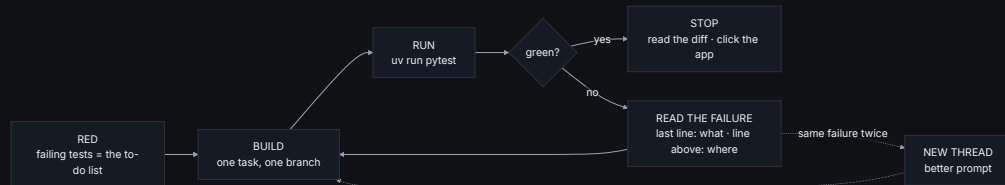
SOMETHING THAT CAN'T LIE
The agent stops when the work looks done. Give it a check that says pass or fail: the spec's Done when and tests/test_feature_2.py. Four red tests are your to-do list.

NEVER MOVE THE GOALPOSTS
An agent that edits a test to make it pass has cheated, not finished. The rule is in AGENTS.md. You enforce it.

EVIDENCE, NOT CLAIMS
"All tests pass" is a sentence. The test output is evidence. Ask for it every time.

GREEN IS NOT THE END
The judge checks what it was told to check. After green: read the diff, click the app. App wrong, tests green? Write down what the judge missed — that's a test that should exist.

Red → build → run → fix → green.



ONE TASK PER LOOP

One bug, or one spec. Not “fix everything you see”. The rule is in the file; the prompt names one file.

LET IT LOOP

Tell it to run the tests after every change and keep going until green. It reads the failure and fixes it. You read the result.

A STOP CONDITION

Green — or a cap. Same failure twice: stop, read it yourself, **new thread**, better prompt.

2026-09-19

Rules, judge, loop

Red → build → run → fix → green.



The three parts are the whole lesson: a rulebook, a judge, a loop. Everything else in agentic coding is a variation. Today the loop runs inside one Codex turn and you watch; the same idea one level up is a script that re-feeds the prompt until the judge passes (the “Ralph loop”). Do not demo the bash loop today unless a laptop has Codex CLI and you have five spare minutes — it is a spectacle, not the lesson.

The loop prompt.

Build `specs/feature-2-categories.md`. Work only on the branch `feature-2-categories`. After every change, run `uv run pytest tests/test_feature_2.py` and keep going until all four tests pass. Do not edit anything in `tests/`. When they pass, paste the final test output and list every file you changed.

THE TASK

One spec, by file name.

THE BOUNDARY

One branch. No `tests/`.

THE JUDGE

The test command, every change.

THE EVIDENCE

Output + file list.

2026-09-19

Rules, judge, loop

└─ The loop prompt.

Four parts, in every loop prompt you ever write: the task, the boundary, the judge, the evidence. Students copy it from the Week 4 page and change two file names. New thread before pasting — the AGENTS.md lines from five minutes ago are only read at thread start.

STANDARD 3 · THE PROMPT

The loop prompt.

Build `specs/feature-2-categories.md`. Work only on the branch `feature-2-categories`. After every change, run `uv run pytest tests/test_feature_2.py` and keep going until all four tests pass. Do not edit anything in `tests/`. When they pass, paste the final test output and list every file you changed.

THE TASK
One spec, by file name.

THE BOUNDARY
One branch. No `tests/`.

THE JUDGE
The test command, every change.

THE EVIDENCE
Output + file list.

Watch it loop. Then don't trust it.

- 1 Branch.** GitHub Desktop → New branch → `feature-2-categories`.
- 2 Red.** `uv run pytest tests/test_feature_2.py` → 4 failed. The to-do list.
- 3 New thread. Paste the loop prompt.** Watch: it edits, runs, reads a failure, edits again. Don't touch anything.
- 4 Green.** Read its output: four passed? Every file it lists?
- 5 Read the diff.** Three places must change: `COLUMNS` in `db.py`, every line of `seed/expenses.csv`, then `uv run python seed.py`. Did it run seed? Did it touch `data/` by hand?
- 6 Click the app.** Add expense → is there a category box? Group page → totals per category? Not tested — only you catch it.
- 7 Commit, push, PR, merge.** Like last week.

2026-09-19

Rules, judge, loop

└─ Watch it loop. Then don't trust it.

Let it loop for real; it usually takes 2-4 rounds. Narrate what you see: "it ran the tests — two still red — it is reading the failure — it edited seed/expenses.csv". The two things it commonly misses: forgetting to run seed.py (data/ still has no category column, so the app crashes on load while the tests stay green, because tests use fresh_data from seed/) and no selectbox in the form (nothing tests app.py). Both are the "green is not the end" lesson — do not fix them silently, show them. If it edits a test: stop, discard, point at the rule, re-prompt.

CLASS 4 · LIVE · FEATURE 2 · 13 MIN

Watch it loop. Then don't trust it.

- 1 Branch. GitHub Desktop → New branch → feature-2-categories
- 2 Red. uv run pytest tests/test_feature_2.py → 4 failed. The to-do list.
- 3 New thread. Paste the loop prompt. Watch: it edits, runs, reads a failure, edits again. Don't touch anything.
- 4 Green. Read its output: four passed? Every file it lists?
- 5 Read the diff. Three places must change: COLUMNS in db.py, every line of seed/expenses.csv, then uv run python seed.py. Did it run seed? Did it touch data/ by hand?
- 6 Click the app. Add expense → is there a category box? Group page → totals per category? Not tested — only you catch it.
- 7 Commit, push, PR, merge. Like last week.

One bug, then feature 2.

SPLITIT

issues/002

→ **feature-2-categories**

Removing a member crashes the page.
Test `test_bug_002`, then categories on
its branch.

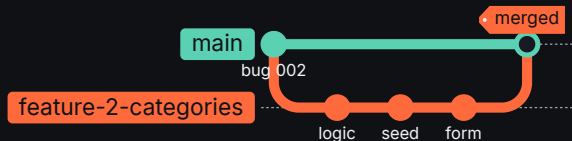
TINY CRM

issues/003

→ **feature-2-conversion**

"Won this month" is always 0. Test
`test_003`, then conversion by source on
its branch.

Bug on `main`. Feature on its own branch, merged by pull request:



2026-09-19

Rules, judge, loop

└ One bug, then feature 2.

Roam for two things: did they ask for the output, and did they read the diff before accepting. Never answer a test failure — "what does the last line say?". CRM 003 has a twist: stamping `closed_on` is not enough; `won_this_month` returns 0 for the whole list when any won lead has no date, and the seed has five. The test tells them; the fix is 'continue', not 'return 0'. SplitIt 002: the test demands a helper called `remove_member_expenses` — read the assertion message. Anyone still on feature 1: that first, then the bug; feature 2 is homework.



Five things you'll see.

- 1 It edited a test.** Discard that file in GitHub Desktop. "Never edit `tests/`." Run again.
- 2 It says done, tests are red.** "Paste the test output." The output is the judge, not the sentence.
- 3 Same failure three times.** Stop. Read it: last line says what, the line above says where. New thread, better prompt.
- 4 It fixed three other things too.** Discard them. One task per loop. Ask again for the one thing.
- 5 Green, but the app is wrong.** The judge missed it. Write down what. Re-prompt with the symptom, or fix by hand.

Every row is a line for your `AGENTS.md`. Keep the list.

2026-09-19

Rules, judge, loop

└ Five things you'll see.

CLASS 4 · WHEN THE LOOP GOES WRONG

Five things you'll see.

- 1** It edited a test. Discard that file in GitHub Desktop. "Never edit `tests/`." Run again.
- 2** It says done, tests are red. "Paste the test output." The output is the judge, not the sentence.
- 3** Same failure three times. Stop. Read it: last line says what, the line above says where. New thread, better prompt.
- 4** It fixed three other things too. Discard them. One task per loop. Ask again for the one thing.
- 5** Green, but the app is wrong. The judge missed it. Write down what. Re-prompt with the symptom, or fix by hand.

Every row is a line for your `AGENTS.md`. Keep the list.

What did it get wrong? Write the rule.

- In your group: the worst thing the agent did today. One sentence each.
- Turn it into a rule. Short, specific, testable. “Run `uv run python seed.py` after changing anything in `seed/`.”
- Add it to `AGENTS.md`. Commit: “*AGENTS.md: <the rule>, because <what happened>*”. Push.
- Three groups read theirs out.

That is how the rulebook grows: from what went wrong, one line at a time. Not from a longer prompt.

2026-09-19

Rules, judge, loop

└─What did it get wrong? Write the rule.

Collect the rules on the board; the good ones become the course’s AGENTS.md for Block 2. Expect: “run seed.py after seed changes”, “don’t touch data/”, “show me the diff before you commit”, “one file at a time”. Reject vague ones (“be careful”) — ask “how would you know it followed this?”.

CLASS 4 · RETRO · 7 MIN

What did it get wrong? Write the rule.

- In your group: the worst thing the agent did today. One sentence each.
- Turn it into a rule. Short, specific, testable. “Run `uv run python seed.py` after changing anything in `seed/`.”
- Add it to `AGENTS.md`. Commit: “*AGENTS.md: <the rule>, because <what happened>*”. Push.
- Three groups read theirs out.

That is how the rulebook grows: from what went wrong, one line at a time. Not from a longer prompt.

Block 2 starts: a spec you build alone.

- **Finish feature 2.** Green, merged, pushed.
- **Your rule is in** `AGENTS.md`, committed, with the reason in the message.
- **Push before you close the laptop.** Next week you build on this repo.

Rulebook · judge · loop.

Everything in Block 2 is those three, on your own spec.

2026-09-19

Rules, judge, loop

└─ Block 2 starts: a spec you build alone.

Nothing to hand in. Next week's check-in: feature 2 merged and a commit on AGENTS.md. Part II of the standards (walking away: scripted loops, goals, review by a second agent) comes with the team project in Block 3.

CLASS 4 · BEFORE NEXT WEEK

Block 2 starts: a spec you build alone.

- Finish feature 2. Green, merged, pushed.
- Your rule is in `AGENTS.md`, committed, with the reason in the message.
- Push before you close the laptop. Next week you build on this repo.

Rulebook · judge · loop.

Everything in Block 2 is those three, on your own spec.